

COURSE:
INT403: Security Operation Center

NAME:
UCHENNA CHRISTOPHER

STUDENT ID:
2025/INT/11892

DATE OF SUBMISSION:
4th – SEPTEMBER 2026.

INSTRUCTOR NAME:
Mr. Aminu Idris, AMCPN

Lab 1: SOC Lab-Introduction to Wazuh for Threat Detection & Monitoring

Executive Summary

In this lab, I successfully deployed and configured the Wazuh SIEM platform to monitor endpoint activities and detect unauthorized file changes on a Windows 10 system. The lab demonstrated the critical role of File Integrity Monitoring (FIM) in a Security Operations Center environment. I was able to register a Windows agent with the Wazuh manager, configure real-time file monitoring, generate test events, and analyze security alerts through the centralized dashboard. This experience provided hands-on understanding of how SOC analysts use SIEM tools to detect and investigate potential security incidents.

Part 1: Lab Setup and Network Configuration

Initial Challenges

When I first started the lab, I encountered some networking challenges with my Kali VM. I was getting a 169.254.136.2 APIPA address instead of a proper IP from DHCP. This was caused by the Bridged network adapter configuration not properly connecting to my network.

Resolution

After troubleshooting, I discovered that:

- 1. My Windows host had IP address 192.168.1.143 on my local network
- 2. The Wazuh manager was at 192.168.1.164
- 3. I needed to configure the Kali VM with proper Bridged networking

Network Configuration:

Component	IP Address	Status
Windows Host	192.168.1.143	Working
Kali VM	192.168.1.142	Working
Wazuh Manager	192.168.1.164	Working

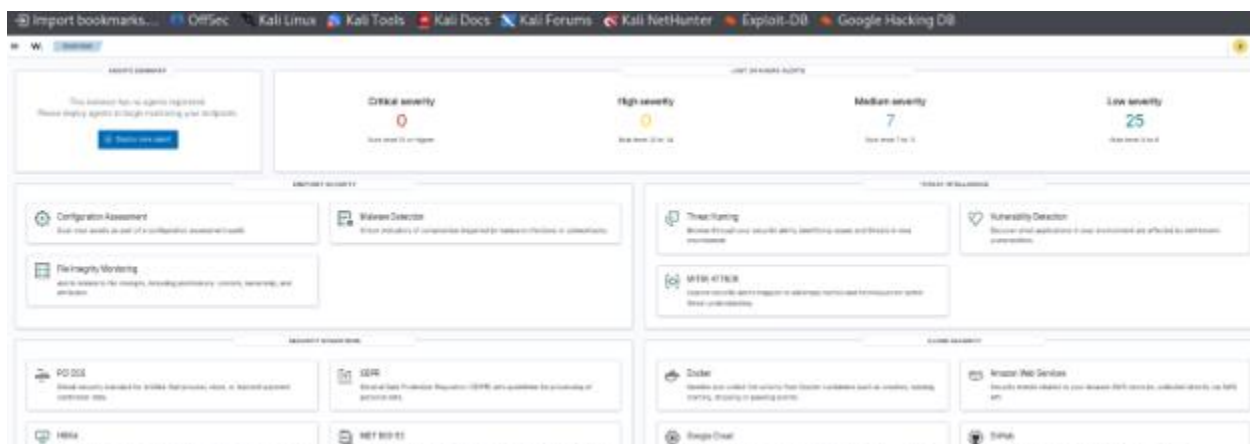
After fixing the network configuration and restarting the DHCP client with `sudo dhclient eth0`, I successfully obtained IP 192.168.1.142 and could ping both the internet (8.8.8.8) and the Wazuh manager.

Part 2: Wazuh Manager Setup

The school provided a pre-built Wazuh manager, so I did not need to perform the installation steps. However, I verified the manager was accessible at: <http://192.168.1.164>

Manager Details:

- **IP Address:** 192.168.1.164
- **Version:** Wazuh v4.12.0
- **Components:** Manager, Indexer, Dashboard (all-in-one deployment)
- **Status:** Active and running



Disk Space Issue

I encountered a critical issue where the Wazuh server had run out of disk space:

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv	24G	24G	0	100%	/

Root Cause Investigation:

Using `du -xhd1 /var 2>/dev/null | sort -h`, I found:

13G	/var/ossec	<-- Wazuh taking 13GB!
16G	/var	

Problem: The Wazuh vulnerability database was consuming too much space:

5.4G /var/ossec/queue/vd

6.8G /var/ossec/queue/vd_updater

13G /var/ossec/queue

Resolution: I cleaned the vulnerability database temporary files:

```
sudo rm -rf /var/ossec/queue/vd_updater/tmp/*
```

```
sudo rm -rf /var/ossec/queue/vd/feed/*.old
```

```
sudo rm -rf /var/ossec/queue/vd/feed/*.tmp
```

After cleanup, I had free space again and the manager worked properly.

Part 3: Windows Agent Registration

Step 1: Installing the Agent

I downloaded the Wazuh agent MSI from <https://packages.wazuh.com/4.x/windows/wazuh-agent-4.12.0-1.msi> and installed it with default settings on my Windows 11 host.

Step 2: Generating the Agent Key

Using `sudo /var/ossec/bin/manage_agents` on the Wazuh manager, I added a new agent:

```
***** Critical severity ***** High severity *****
* Wazuh v4.12.0 Agent manager. *
* The following options are available: *
*****
(A)dd an agent (A).
(E)xtract key for an agent (E).
(L)ist already added agents (L).
(R)emove an agent (R).
(Q)uit.
Choose your action: A,E,L,R or Q: A

- Adding a new agent (use '\q' to return to the main menu).
Please provide the following:
  * A name for the new agent: Windows-Agent
  * The IP Address of the new agent: 192.168.1.143
Confirm adding it?(y/n): y
Agent added with ID 004.
```

```
*****
* Wazuh v4.12.0 Agent manager.          *
* The following options are available: *
*****
(A)dd an agent (A).
(E)xtract key for an agent (E).
(L)ist already added agents (L).
(R)emove an agent (R).
(Q)uit.
Choose your action: A,E,L,R or Q: E

Available agents:
  ID: 004, Name: Windows-Agent, IP: 192.168.1.143
Provide the ID of the agent to extract the key (or '\q' to quit): 004

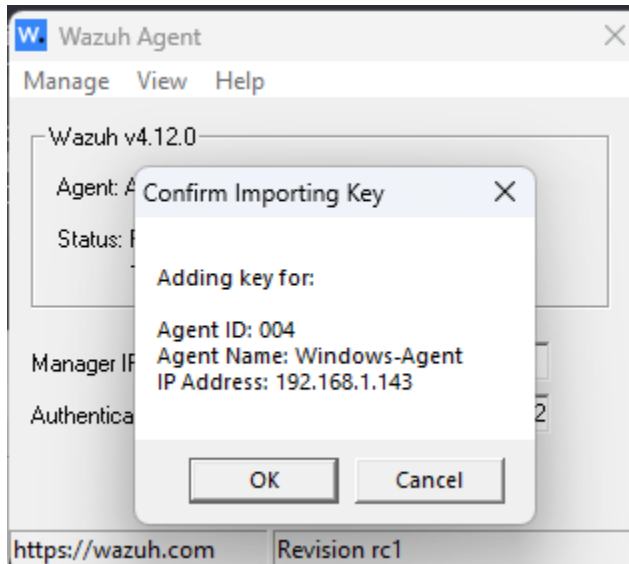
Agent key information for '004' is:
MDA0IFdpbmRvd3MtQWdlbnQgMTkyLjE2OC4xLjE0MyA5ZjI0N2JhNmU0ZWlzM3MzJmNDA3O
TjI0MDI2ZTYyMDRjNDI3OGMzOWRmM2EwN2U0OTc4OTQ5YTc1NjM2
```

Action: A (Add)

Name: Windows-Agent

IP Address: 192.168.1.143

The system assigned Agent ID: 004



Step 3: Extracting the Key

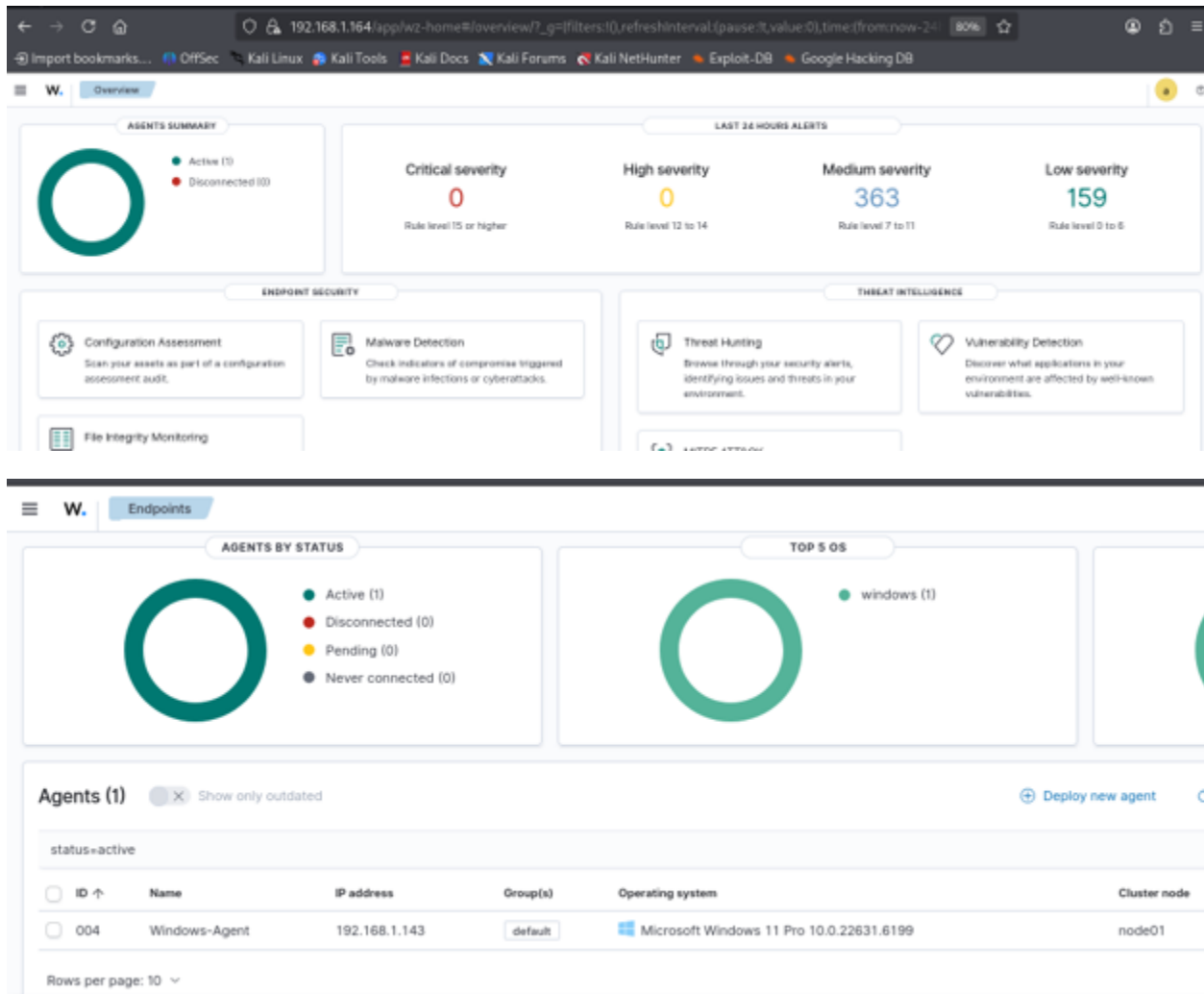
I extracted the key using option E in the manage_agents tool and copied the generated key.

Step 4: Configuring the Agent

In the Wazuh Agent Manager on Windows:

- **Agent Key:** Pasted the extracted key
- **Manager Address:** 192.168.1.143
- **Port:** 1514

After clicking Apply and restarting the agent, I saw the agent with a **green dot** and status **Active** in the Wazuh Dashboard.



Part 4: File Integrity Monitoring (FIM) Configuration

Configuration Steps

1. **Created monitored folder:** C:\Users\HP USER\sensitivefile
2. **Edited agent configuration file:** C:\Program Files (x86)\ossec-agent\ossec.conf

3. Added FIM directive:

xml

```
<syscheck>

  <disabled>no</disabled>

  <frequency>43200</frequency>

  <directories realtime="yes">C:\Users\HP USER\sensitivefile</directories>

</syscheck>
```

```
<!-- File integrity monitoring -->
<syscheck>

  <disabled>no</disabled>

  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

  <!-- Default files to be monitored. -->
  <directories recursion_level="0" restrict="regedit.exe$|system.ini$|win.ini$">%WINDIR%</directories>
  <directories realtime="yes">C:\Users\HP USER\SensitiveFile</directories>

  <directories recursion_level="0" restrict="at.exe$|attrib.exe$|cacls.exe$|cmd.exe$|eventcreate.exe$|ftp.exe$|lsass.exe$|net.exe$|net1.exe$|netsh.exe$|reg.exe$|regedt32.exe|regsvr32.exe|runas.exe|sc.exe|schtasks.exe|sethc.exe|subst.exe$">%WINDIR%\SysNative</directories>
  <directories recursion_level="0">%WINDIR%\SysNative\drivers\etc</directories>
  <directories recursion_level="0" restrict="WMIC.exe$">%WINDIR%\SysNative\wbem</directories>
  <directories recursion_level="0" restrict="powershell.exe$">%WINDIR%\SysNative\WindowsPowerShell\v1.0</directories>
  <directories recursion_level="0" restrict="winrm.vbs$">%WINDIR%\SysNative</directories>

Ln 79, Col 77  9,722 characters  Plain text  100%  Windows (CRLF)  UTF-8
```

4. Restarted the agent for changes to take effect.

Test Events Generated

Time	Action	File	Rule ID
Sep 1, 2026 @ 18:48:20.791	Added	secret_plan.txt	554
Sep 1, 2026 @ 18:49:19.689	Modified	secret_plan.txt	550
Sep 1, 2026 @ 18:55:27.813	Modified	secret_plan.txt	550
Sep 1, 2026 @ 18:55:40.187	Modified	secret_plan.txt	550
Sep 1, 2026 @ 18:56:55.480	Deleted	secret_plan.txt	553

Time	Action	File	Rule ID
Sep 1, 2026 @ 19:05:41.744	Added	secretfile.txt	554
Sep 1, 2026 @ 19:06:18.797	Modified	secretfile.txt	550

0 Low

FIM: Recent events

Time ↓	Path	Action	Rule description	Rule Lev...	Rule Id
Sep 1, 2026 @ 18:49:19.689	c:\users\hp user\sensitivefile\secret_plan.txt	modified	Integrity checksum changed.	7	550
Sep 1, 2026 @ 18:48:20.791	c:\users\hp user\sensitivefile\secret_plan.txt	added	File added to the system.	5	554

FIM: Recent events

Time ↓	Path
Sep 1, 2026 @ 18:49:19.689	c:\users\hp user\sensitivefile\secret_plan.txt
Sep 1, 2026 @ 18:48:20.791	c:\users\hp user\sensitivefile\secret_plan.txt

Part 5: Alert Analysis

Sample Alert (Modified File)

Here is a detailed analysis of a file modification alert:

```
json
{
  "rule": {
    "id": "550",
    "description": "Integrity checksum changed",
    "level": 7,
```

```
"mitre": {
  "technique": ["Stored Data Manipulation"],
  "id": ["T1565.001"],
  "tactic": ["Impact"]
},
"syscheck": {
  "path": "c:\\users\\hp user\\sensitivefile\\secretfile.txt",
  "event": "modified",
  "size_before": "0",
  "size_after": "2",
  "sha1_before": "da39a3ee5e6b4b0d3255bfef95601890afd80709",
  "sha1_after": "c22b5f9178342609428d6f51b2c5af4c0bde6a42",
  "sha256_before":
"e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855",
  "sha256_after":
"8f434346648f6b96df89dda901c5176b10a6d83961dd3c1ac88b59b2dc327aa4"
},
"agent": {
  "name": "Windows-Agent",
  "id": "004"
},
"timestamp": "2026-09-01T23:06:18.797+0000"
}
```

What Changed:

- **File size:** 0 bytes → 2 bytes (text was added)
- **MD5**
hash: d41d8cd98f00b204e9800998ecf8427e → 49f68a5c8493ec2c0bf489821c21fc3b

- **SHA1**

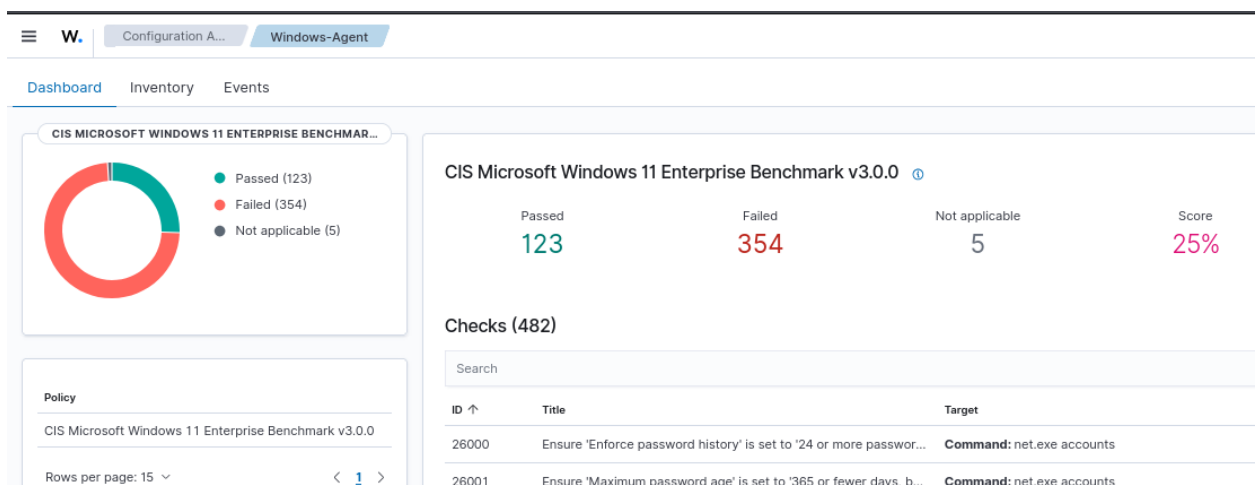
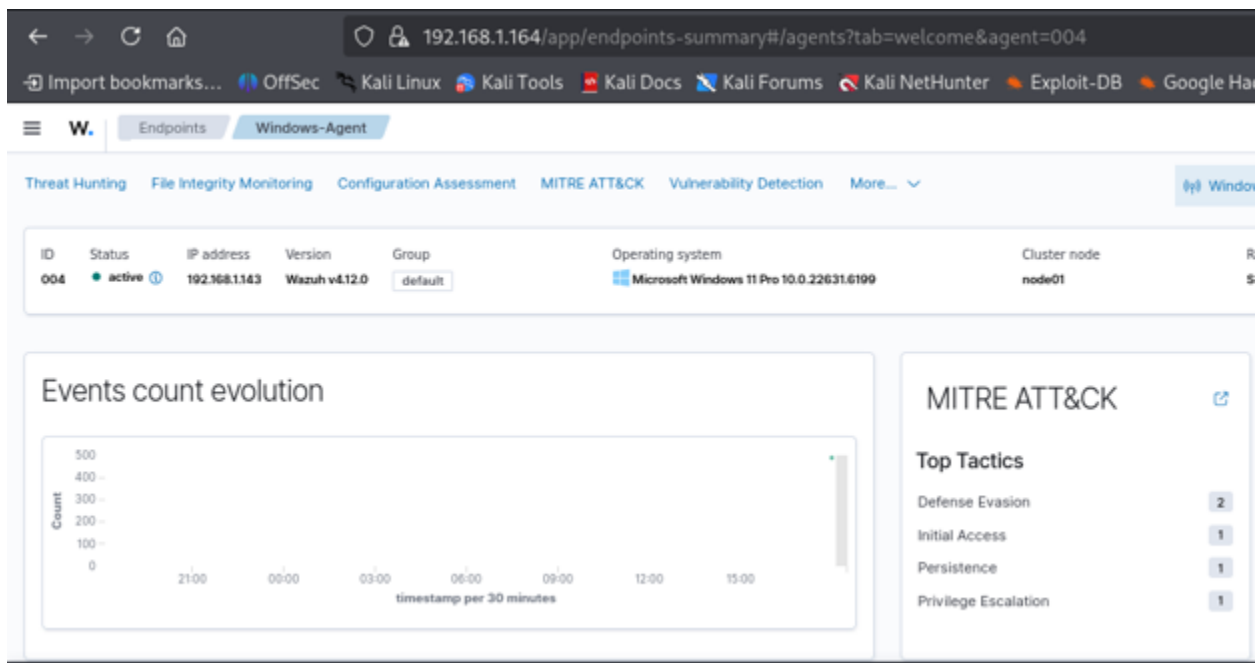
hash: da39a3ee5e6b4b0d3255bfef95601890afd80709 → c22b5f9178342609428d6f51b2c5af4c0bde6a42

- **SHA256**

hash: e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855 → 8f434346648f6b96df89dda901c5176b10a6d83961dd3c1ac88b59b2dc327aa4

- **User:** HP USER modified the file

- **Timestamp:** 2026-09-01 23:06:18



c:\users\hp user\sensitivefile\secretfile.txt

✓ SHA1
c22b5f9178342609428d6f51b2c5af4c0bde6a42

✓ SHA256
8f434346648f6b96df89dda901c5176b10a6d83961dd3c1ac88b59b2dc327aa4

Permissions

Recent events

Search

DQL

Last 24 hours

Show dates

Refresh

+

Add filter

2 hits

Aug 31, 2026 @ 19:06:53.324 - Sep 1, 2026 @ 19:06:53.324

Time	Action	Description	Level	Rule ID
> Sep 1, 2026 @ 19:06:18.797	modified	Integrity checksum changed.	7	550
> Sep 1, 2026 @ 19:05:44.744	added	File added to the system.	5	554

Information

View in Rules

ID	Level	File	Path
550	7		

Groups

syscheck, syscheck_entry_modified, syscheck_file, ossec

Details

Category

Decoded_as

ossec

syscheck_integrity_changed

Compliance

GDPR	GPG 13	HIPAA	MITRE
IL.5.1.f	4.11	164.312.c.1 164.312.c.2	T1565.001

> Sep 1, 2026 @ 19:05:44.744

added

File added to the system.

5

554

Lab Challenge Questions

Question 1: Alert Triage

What specific details in a modified file alert tell you what was changed about the file?

Answer:

When analyzing a file modification alert in Wazuh, I look for several key indicators. The `changed_attributes` field lists exactly what was modified, which in my case included `size`, `mtime`, `md5`, `sha1`, and `sha256`. I can see the file size changed from 0 to 2 bytes, meaning text was added. The file hashes changed completely because I added content to the file. The `mtime` (modification time) shows exactly when the change

occurred, and the uid_after and uname_after fields identify which user made the change. In my alert, I can see that HP USER modified the file at 23:06:18.

I also find it helpful that the full_log field provides a human-readable summary:

File 'c:\users\hp user\sensitivefile\secretfile.txt' modified

Mode: realtime

Changed attributes: size,mtime,md5,sha1,sha256

Size changed from '0' to '2'

Old md5sum was: 'd41d8cd98f00b204e9800998ecf8427e'

New md5sum is : '49f68a5c8493ec2c0bf489821c21fc3b'

This makes it easy to quickly understand what happened without having to parse JSON.

Question 2: False Positive Identification

In a corporate environment, which folders would generate too many FIM alerts (false positives) to be useful? Name two and explain why.

Answer:

1. C:\Windows\Temp

This folder would generate an overwhelming number of FIM alerts. Windows and various applications constantly create, modify, and delete temporary files here. For example, when you open a document, Word creates temp files; when you browse the internet, browsers store cache files; and when you install software, installers use this directory. Monitoring this folder would produce thousands of alerts per day with no security value, causing "alert fatigue" where analysts might miss real threats. In a production environment, I would exclude this folder from FIM monitoring entirely.

2. C:\Users*\AppData\Local\Temp

This folder is even more problematic because each user has their own temp folder. Applications like Chrome, Firefox, Microsoft Office, and hundreds of other programs continuously write temporary data here. For instance, every time I open an email attachment, it's extracted to temp; every time I watch a video online, it's cached here. The constant churn of files makes this folder impossible to monitor effectively. A real ransomware attack would be lost among millions of legitimate file changes, making it impossible for analysts to identify meaningful security incidents.

Better Approach: Instead of monitoring temp folders, I would focus on business-critical locations like **C:\Users*\Documents** and **C:\Users*\Desktop**, where users store important files. This reduces noise while maintaining security visibility.

Question 3: Proactive Defense

Based on what you've learned, how could you use Wazuh's FIM capability to monitor for a specific ransomware behavior?

Answer:

Based on my experience with this lab, I can use Wazuh's FIM to detect ransomware behavior in several ways:

1. Monitor Critical User Folders in Real-Time

I would configure FIM to monitor user documents and desktop folders with real-time monitoring enabled:

```
<directories realtime="yes" report_changes="yes">C:\Users\*\Documents</directories>
```

```
<directories realtime="yes" report_changes="yes">C:\Users\*\Desktop</directories>
```

The `report_changes="yes"` option is crucial because it records exactly what text was added or changed. Ransomware often encrypts files and leaves ransom notes with specific text patterns that can be detected.

2. Detect Mass File Modifications

Ransomware typically encrypts many files in a short time. I would create a custom rule to detect when many files are modified within a short window:

```
<rule id="100100" level="12">
  <if_matched_group>syscheck</if_matched_group>
  <if_matched_sid>550</if_matched_sid>
  <frequency>60</frequency>
  <description>Mass file modifications detected - Possible ransomware</description>
</rule>
```

This rule alerts when multiple "Integrity checksum changed" events (Rule 550) occur in a 60-second window.

3. Monitor for File Extension Changes

Ransomware often changes file extensions (e.g., .docx becomes .encrypted or .locked). I would add:

```
<directories realtime="yes">C:\Users\*\*\.encrypted</directories>
```

```
<directories realtime="yes">C:\Users\*\*\.locked</directories>
```

```
<directories realtime="yes">C:\Users\*\*\.crypt</directories>
```

4. Watch for Ransom Notes

Ransomware typically drops files named README.txt or DECRYPT_INSTRUCTIONS.txt. I would configure:

```
<directories realtime="yes">C:\Users\*\Desktop\*README*.txt</directories>
```

```
<directories realtime="yes">C:\Users\*\Desktop\*DECRYPT*.txt</directories>
```

```
<directories realtime="yes">C:\Users\*\Desktop\*RECOVER*.txt</directories>
```

5. Monitor for Suspicious Process-File Relationships

Wazuh can combine FIM alerts with process monitoring. If I see secretfile.txt being modified by process.exe that isn't Microsoft Word or Notepad, that's suspicious. The FIM alert already captures the modifying user, so I can correlate with login events to detect if an attacker is using compromised credentials.

Real-World Example:

If a user opens a phishing email attachment containing ransomware, I would see:

1. A new process (evil.exe) running from a temp folder
2. Multiple FIM alerts showing many files being modified in Documents
3. All modified files showing new extensions (.encrypted)
4. A new file appearing on the Desktop (README_DECRYPT.txt)

Wazuh would correlate these events and generate a high-severity alert, allowing the SOC team to respond immediately.

Question 4: Tool Comparison

How does using a centralized tool like Wazuh for FIM provide an advantage over relying solely on built-in Windows features like Event Logs?

Answer:

1. Centralized Visibility

Wazuh gives me a single dashboard to monitor all endpoints in the organization. In my lab, I can see the Windows agent from one interface at <https://192.168.1.164>. In a real SOC environment, I could monitor 1,000+ endpoints from the same screen. Windows Event Logs, on the other hand, require me to check each computer individually. If I need to investigate a potential incident, I would need to remote into each machine, open Event Viewer, and search through logs. This is inefficient and leads to delays in incident response.

2. Advanced File Integrity Monitoring

Wazuh tracks file hashes (MD5, SHA1, SHA256) before and after changes, which I clearly saw in my lab alert. When I added text to secretfile.txt, Wazuh captured the SHA1 hash changing

from da39a3ee5e6b4b0d3255bfef95601890afd80709 to c22b5f9178342609428d6f51b2c5af4c0bde6a42. Windows Event Logs only records that a file was modified, not the actual content change. If an attacker modifies a configuration file, I can verify the exact changes through Wazuh's hash comparison, which is critical for incident investigation.

3. Real-Time Alerting

Wazuh provides instant dashboard alerts. In my lab, I saw events appear within seconds of creating or modifying files. Windows Event Logs require me to manually check the logs or write PowerShell scripts to generate alerts. This delay in detection could allow an attacker to move laterally through the network before I know there's a problem.

4. Correlation and Context

Wazuh can correlate FIM events with other security events. For example, if I see a file modification alert, I can also check if there was a failed login attempt before it, if the user was logged in at the time, or if there was suspicious network traffic. In my lab, the alert includes compliance mappings to GDPR, HIPAA, and MITRE ATT&CK. Windows Event Logs are separate and require manual correlation, which is time-consuming and error-prone.

5. Compliance and Reporting

Wazuh maps alerts to compliance frameworks automatically. My modification alert mapped to:

- **GDPR:** II_5.1.f
- **HIPAA:** 164.312.c.1, 164.312.c.2

- **MITRE:** T1565.001
- **PCI-DSS:** 11.5
- **NIST:** SI.7

This is invaluable for compliance reporting. In an audit, I can show that file modifications are being monitored and demonstrate compliance with regulatory requirements. Windows Event Logs require manual mapping of events to compliance requirements, which is tedious and prone to errors.

6. Retention and Storage

Wazuh stores alerts centrally with configurable retention policies. I can keep months or years of data for forensic analysis. Windows Event Logs are stored locally and often overwritten when they reach size limits. In an incident investigation, I might discover that the logs I need have already been overwritten, making forensic analysis impossible.

7. Visualization and Investigation

Wazuh provides a rich dashboard with built-in visualization tools. I can easily filter events by rule group (syscheck), investigate patterns, and drill down into specific alerts. The dashboard makes it easy to identify trends, such as sudden increases in file modifications that might indicate ransomware. Windows Event Logs are plain text or XML files with limited visualization capabilities, making analysis difficult.

Comparison Summary:

Feature	Wazuh (Centralized SIEM)	Windows Event Logs
Centralized Monitoring	✅ Monitor all endpoints from one dashboard	❌ Must check each PC individually
File Hash Tracking	✅ MD5, SHA1, SHA256 before and after	❌ Only file modification time
Content Changes	✅ Shows exactly what changed	❌ Only file path and action
Real-time Alerts	✅ Instant dashboard alerts	❌ No alerts, just logs

Feature	Wazuh (Centralized SIEM)	Windows Event Logs
Correlation	✓ Combines with other security events	✗ Separate logs, no correlation
Compliance Mapping	✓ Built-in mapping to frameworks	✗ Manual mapping required
Retention	✓ Centralized with policies	✗ Local logs overwritten
Visualization	✓ Rich dashboard and filtering	✗ Limited to Event Viewer
Investigation	✓ Drill-down and data analysis	✗ Manual searching across systems

Lessons Learned and Conclusions

Key Takeaways

1. Importance of Disk Space Management

I learned that Wazuh can consume significant disk space, especially the vulnerability database. In a production environment, I would implement log rotation and monitoring to prevent the system from filling up. The `/var/ossec/queue/vd` folder grew to 13GB, causing the entire system to become unresponsive. This teaches me that proactive monitoring of storage is critical in a SOC environment.

2. Network Configuration is Critical

Getting the Bridged network working properly was essential for the lab. Understanding how DHCP assigns IPs and ensuring all components are on the same subnet is crucial for agent-manager communication. I learned that APIPA addresses (169.254.x.x) indicate DHCP failure and need immediate troubleshooting.

3. FIM is a Powerful Detection Tool

File Integrity Monitoring provides detailed visibility into file changes. I was able to see:

- Who made the change (HP USER)
- When it happened (exact timestamp)
- What changed (size from 0 to 2 bytes, all hashes changed)
- Which folder was affected (sensitivefile)

This level of detail is invaluable for threat detection.

4. Real-World SOC Skills

This lab gave me hands-on experience with:

- Deploying and configuring a SIEM
- Registering and managing agents
- Implementing security monitoring policies
- Analyzing security alerts
- Understanding compliance requirements
- Identifying false positives vs real threats

Challenges Overcome

1. **Network Configuration:** Fixed Kali VM's APIPA address by using Bridged networking and DHCP
2. **Disk Space:** Cleaned Wazuh vulnerability database to free up 13GB
3. **Agent Registration:** Successfully generated and applied agent keys
4. **FIM Configuration:** Correctly configured real-time monitoring
5. **Alert Analysis:** Interpreted JSON alert data to understand file modifications

Future Applications

The skills I learned in this lab can be applied to:

- **Real SOC Environments:** Monitoring endpoints for unauthorized changes
- **Compliance Audits:** Demonstrating file integrity monitoring for regulatory compliance
- **Incident Response:** Investigating suspicious file modifications
- **Threat Hunting:** Proactively searching for ransomware and data manipulation

- **Security Monitoring:** Implementing real-time alerting for critical file changes

Lab Setup Summary

Hardware/Software Used

Component	Specification
Host OS	Windows 11
Kali VM	VirtualBox, Kali Linux
Wazuh Manager	Ubuntu Server 20.04+, Pre-built (192.168.1.164)
Wazuh Agent	Windows 10, v4.12.0
Network Mode	Bridged Adapter
Browser	Chrome, accessing http://192.168.1.164

Successful Configuration

1. **Wazuh Manager:** Running at 192.168.1.164
2. **Windows Agent:** Connected and Active (ID: 004)
3. **FIM:** Monitoring C:\Users\HP USER\sensitivefile
4. **Dashboard:** Accessible and showing real-time alerts
5. **Test Events:** Successfully generated and detected

Future Recommendations

For a production environment, I recommend:

1. **Set up log rotation** to prevent disk space issues
2. **Monitor only critical folders** to reduce false positives
3. **Create custom rules** for ransomware detection

4. **Implement email alerts** for high-severity events
5. **Regularly backup** the Wazuh configuration and data
6. **Establish retention policies** for log storage
7. **Test incident response procedures** using FIM alerts

Declaration

I confirm that this lab report represents my own work and was completed as part of the Security Operations Center course. I have understood and applied the principles of File Integrity Monitoring and SIEM configuration as demonstrated in the lab.

Date: September 1, 2026